DO CONTEXT ENGINES HELP? RETRIEVAL QUALITY, DETERMINISM, AND AGENT UTILITY

Anonymous authors
Paper under double-blind review

ABSTRACT

Coding agents are increasingly wrapped in "context engines" that index a repository and return the files the model might need. It is not obvious that this helps: a competent agent already has `grep`, and hosted retrievers are often slower and less inspectable than a local search. We treat the question as a measurement problem and run it to completion under a preregistered protocol: fresh exposure-accounted splits, canonical git corpora, frozen code for every confirmatory number, exact attempt accounting for hosted engines, and cluster-aware uncertainty for every claim. On the untouched Agent Retrieval Bench round-3 final (220 cases, 25 repositories), the frozen Delphi stack reaches MRR 0.332 and Recall@20 0.648, beating whole-file BM25, the official lexical ranker, and their tuned reciprocal-rank fusion on all four preregistered metrics with repository-cluster 95% intervals excluding zero. On 62 held-out SWE-bench Verified localization instances it leads MRR decisively (0.680 vs. 0.406) and ties Recall@20. On documentation retrieval we show that the apparent quality lead of a hosted retrieval-plus-synthesis engine (0.575 vs. 0.400 identifier hit) is an output-contract artifact: routing every engine's retrieved context through the same frozen synthesis stage yields a three-way statistical tie (0.617/0.625/0.575) barely above a no-retrieval floor of 0.492, at less than half the hosted engine's latency. Delphi is near-deterministic where hosted engines are not (98.0% vs. 35.6% and 0.0% pairwise exact context). A closed-tool agent intervention shows retrieval seeds causally change what a frozen agent submits; direct DS-1000 utility deltas remain unresolved at $n{=}40$ for every engine. We do not claim state of the art: the hosted engine's repository arm never reached required coverage (32/68 snapshots; a minimal 1.5 MB probe stalls in ingestion), and our claim rule requires the complete comparable engine set. The paper reports the protocol, the numbers, the failure modes that actually changed the product, and what remains blocked.

1 INTRODUCTION

The pitch for a context engine is simple. Index the repo. Embed the chunks. When the agent (or the human) asks a question, return the files that matter. The pitch for *not* building one is simpler: `rg` is deterministic, takes 30 milliseconds, and already found `src/_pytest/assertion/rewrite.py` when we asked where pytest rewrites asserts.

We built Delphi as a local-first retrieval stack—hybrid lexical/vector search, query expansion, a cross-encoder, an optional listwise rerank—and then watched it lose to a hosted engine and to grep in sloppy internal benches. Those benches mixed embedding spaces, leaked gold into queries, and treated a single run as a fact. This paper is the repair, run to completion: a locked protocol, fresh splits, official metrics over canonical git text, a variance study that ends in an engineering fix rather than a shrug, matched-contract comparisons against two hosted engines on their strongest surface, a closed-tool agent intervention, and one-shot confirmatory finals from frozen code.

Three research questions:

1. **RQ1 (utility).** Does a context seed change what a frozen coding agent submits, relative to no seed, a grep-style seed, an oracle seed, and a random-file seed? Does any engine's context measurably improve downstream code generation?
2. **RQ2 (ranking).** Is Delphi state of the art among directly comparable engines on agent-relevant repository retrieval?
3. **RQ3 (determinism).** Where does run-to-run variance come from, how large is each source, and what does removing it cost?

Short answers. **RQ2:** on every runnable comparison Delphi now leads or ties—decisively against the strongest local baselines on the untouched finals—but the hosted engine's repository arm never became scoreable, so "state of the art" stays unclaimed by our own rule. **RQ3:** the variance was ours, not the embedding API's; total SQL ordering, single-flight sharing of hosted calls, and a persistent stage cache take exact repeat rates from 30.6% to 100% in-process and make results restart-durable. **RQ1:** seeds causally move a frozen agent (a July Delphi seed nearly triples file-F1 over no seed), but on DS-1000 no engine's context—ours included—separates from a no-retrieval control at $n{=}40$. The rest of the paper is the evidence.

2 Related work

SWE-bench established executable repository-level issue resolution (Jimenez et al., 2024), while SWE-agent and Agentless expose repository navigation and localization as explicit stages (Yang et al., 2024; Xia et al., 2024). LocAgent studies a graph-guided localization agent and reports that better localization can improve multi-attempt repair (Chen et al., 2025). These systems remain end-to-end or localization pipelines: a failed patch can still reflect retrieval, reasoning, editing, or validation.

Repository-level code completion provides complementary evidence that cross-file retrieval matters once the edit location is known. RepoCoder alternates retrieval and generation (Zhang et al., 2023); CodeRAG adds multi-path retrieval and preference-aligned reranking (Zhang et al., 2025). Dense retrieval, late interaction, and hypothetical-document expansion motivate components used by practical context engines (Karpukhin et al., 2020; Khattab & Zaharia, 2020; Gao et al., 2023), while BM25 remains a strong exact-term baseline for code (Robertson & Zaragoza, 2009).

The closest work now measures context acquisition directly. ContextBench provides human-verified file, block, and line contexts for 1,136 issue-resolution tasks and scores agent trajectories (Li et al., 2026). CORE-Bench scales requirement-driven retrieval across code understanding, edit localization, and broader context, and finds a large drop from traditional code search to repository-state retrieval (Zhang et al., 2026a). SWE-Explore derives audited line-level targets from successful trajectories and validates exploration metrics with a restricted-context repair bridge (Zhang et al., 2026b). Agent Retrieval Bench contributes workflow-derived next-context targets, base-commit hygiene, and budget-aware file metrics (Qin & Xie, 2026). We build on this evaluation line rather than claiming a new retrieval task: our focus is the intervention itself—holding the agent and tool policy fixed while varying only its initial context—plus repeated-system determinism, matched output contracts across hosted engines, and development-to-confirmatory product iteration.

3 Protocol

Exposure and splits. All 427 ARB v2 cases were scored in a July 2026 internal round. Round 3 therefore draws a new split with salt `delphi-round3-v1` (25% development / 75% confirmatory within each `(release, repo)` stratum) and excludes 50 `legacy_exposed` trace2code IDs. Development ($n{=}75$ positive cases, 68 snapshots) may guide product changes; the confirmatory final ($n{=}220$ positive cases, 142 snapshots) is queried exactly once after freeze. Two further finals have never been used for development: an independent commit-to-files split over a repository-disjoint corpus (18 cases, 48 snapshots), and 62 stratified SWE-bench Verified localization instances (12 repositories; a leakage audit finds patch markers in $0/62$ queries). A 40-case DS-1000 development subset (8 cases per library over Matplotlib, NumPy, pandas, scikit-learn, TensorFlow) drives the documentation and downstream tracks.

Queries, corpus, metrics. Queries are the official ARB gold-free structured objects, serialized with sorted keys—not free-text paraphrases. File text comes from `git show` against bare clones at each case's `base_commit`; binary and non-UTF-8 blobs are skipped. Metrics are the official ARB `sample_metrics` (MRR, Recall@k, $F_{0.5}$@k) plus BCY@8k, the budget-constrained yield from packing ranked files into an 8,000-token context. Documentation cases score *identifier hit*: whether the returned context contains the case's gold API identifier. Downstream utility is DS-1000 pass@1 under the official execution harness in an isolated interpreter.

Hosted-engine accounting. Before scoring a hosted engine, a coverage preflight requires a source for every sampled `(repo, base_commit)` pair; partial runs abort and cannot enter a table. We added this check after catching a manifest whose row count looked complete but whose snapshot intersection was not. Every hosted run records exact attempt counts (one HTTP attempt per observation unless stated), latencies, and response hashes; balanced acquisition (alternating arms within case) is used wherever a hosted provider's drift over time could masquerade as a treatment effect.

Preregistration and freeze. Hypotheses, success criteria, and analysis plans are recorded in a persistent research map before execution; failed candidates stay recorded. All confirmatory numbers come from one frozen commit (an auditable branch equal to the development winner plus one cherry-picked indexing fix, §7), with exact vector scan, API top-k=20, response-level serving-configuration attestation, and a per-gold-path searchability preflight on every provisioned index. Paired deltas carry cluster bootstrap 95% intervals (20,000 resamples, seed 1042; repository clusters for repository tracks, case clusters for documentation and generation), plus exact McNemar tests on any-gold@20 movement.

Claim rule. "State of the art" is reserved for a named track where Delphi leads every directly comparable engine on the untouched primary metric, or is statistically tied while leading a predeclared complementary metric, with the confirmatory partition unused for development *and the complete comparable engine set runnable*. Losses are reported with the same prominence as wins.

4 RQ3: DETERMINISM, FROM DIAGNOSIS TO FIX

Embeddings are not the problem. We embedded the same texts N=20 times per provider (Table 1). OpenAI embeddings jitter at the fifth decimal; the small model never changed top-ten membership or order, and Gemini was bitwise deterministic.

Table 1: Embedding-level repeats (N=20). Ranking is over the cosine neighborhood of each text against a fixed gallery.

Provider	Exact-equal	Max cosine dist.	Top-10 identical
`text-embedding-3-small`	69.3%	2.3×10^{-4}	100%
`text-embedding-3-large`	80.6%	5.1×10^{-5}	98.7%
`gemini-embedding-001`	100%	≈ 0	100%

The pipeline was. Repeating eight ARB queries ten times against the July stack gave a mean exact top-10 list rate of 30.6% (Jaccard@10 0.760, Kendall τ 0.959). Paired traces isolated three causes: uncached, unseeded LLM stages (query expansion, listwise rerank); insertion-order ties in fusion SQL; and approximate HNSW under concurrency. The fixes are unglamorous: total SQL ordering on every branch, single-flight sharing of hosted call outcomes across concurrent identical requests, an explicit LLM seed, and an optional SQLite-backed persistent cache for expansion, listwise, and embedding outputs. After the fixes, exact and HNSW variants both produce 100% exact top-10 lists across two concurrent 8-query×10-repeat packets and two concurrent 75-case repeats. A genuine API cold restart still moved lists (1/8 exact) because hosted stages resample across process lifetimes; with the persistent cache warm, 8/8 lists survive two real restarts (24/24 pairwise), and mean packet latency falls from 4.90 s on fill to 1.48–1.79 s. We claim restart-durable repeatability from the first persisted answer, not that two clean installations agree on their first answer.

Table 2: ARB round-3 final: 220 untouched cases, scored once from frozen code. Comparators run over the identical canonical-git corpus. Bracketed values are paired repository-cluster bootstrap 95% intervals on the Delphi−baseline delta; any-gold@20 movement is recovered/lost with exact McNemar p.

System	MRR	R@5	R@20	BCY@8k
Delphi (frozen)	**0.332**	**0.476**	**0.648**	**0.296**
Lexical+BM25 RRF	0.160	0.204	0.518	0.124
Lexical ranker	0.144	0.215	0.478	0.138
BM25 (whole-file)	0.128	0.177	0.425	0.075
Δ vs. RRF	+0.172 [0.083, 0.235]	+0.272 [0.148, 0.372]	+0.130 [0.024, 0.200]	+0.172 [0.098, 0.263]
Δ vs. lexical	+0.188 [0.106, 0.246]	+0.261 [0.140, 0.361]	+0.170 [0.088, 0.226]	+0.158 [0.076, 0.255]
Δ vs. BM25	+0.204 [0.111, 0.277]	+0.299 [0.157, 0.433]	+0.223 [0.111, 0.310]	+0.221 [0.129, 0.303]
any-gold@20	Delphi 163/220; vs. RRF $+51/-19$ (p=1.7×10^{-4}); vs. lexical $+56/-16$ (p=2.4×10^{-6}); vs. BM25 $+63/-13$ (p=5.0×10^{-9})			

Table 3: Generalization finals from the same frozen commit. Independent: 18 repository-disjoint commit-to-files cases. SWE-bench: 62 Verified localization instances. Ties are reported as ties.

Track	System	MRR	R@5	R@20	BCY@8k
SWE-bench (62)	**Delphi (frozen)**	**0.680**	**0.704**	**0.737**	**0.638**
	Lexical ranker	0.406	0.509	0.702	0.387
	Lexical+BM25 RRF	0.354	0.500	0.719	0.306
	BM25 (whole-file)	0.149	0.230	0.416	0.145
Independent (18)	**Delphi (frozen)**	**0.508**	**0.551**	0.750	**0.454**
	Lexical+BM25 RRF	0.473	0.500	0.759	0.287
	Lexical ranker	0.438	0.403	0.704	0.259
	BM25 (whole-file)	0.370	0.458	0.676	0.259

Hosted engines, same probe. On ten documentation cases × ten repeats, Delphi returns byte-identical context in 98.0% of run pairs (identifier-hit agreement 1.000); the documentation-only hosted engine manages 35.6% (quality-guided) and 25.3% (oracle IDs), and the retrieval-plus-synthesis engine never repeats a byte (0.0%), though its underlying five-citation sets are near-stable (mean Jaccard 0.9933). An audit also caught resolver-identity drift in the documentation engine: the same library resolved to different corpus IDs across periods, which silently mixes resolver and retrieval effects unless IDs are pinned—so all our comparisons pin them.

5 RQ2A: REPOSITORY RETRIEVAL ON THE UNTOUCHED FINALS

Development, briefly. The development partition drove a sequence of preregistered ablations whose outcomes we keep regardless of direction: exact vector scan replaced HNSW (+0.020 Recall@20 [0.000, 0.063], two any-gold recoveries, no losses); query expansion stayed on (three complete-miss recoveries vs. one loss); a Gemini embedding swap was rejected (MRR −0.054 [−0.102, −0.011]); retrieval top-k was locked at 20; and a file-level Okapi BM25 branch—attractive in storage terms after a 26.7× compaction—was rejected when its single approved configuration failed preregistered adoption gates (MRR interval floor −0.050 and BCY floor −0.037 below the −0.01 bound; median latency ratio $1.342 > 1.2$) despite repeating exactly and losing zero any-gold cases. It ships default-off.

Final. Table 2 is the answer to RQ2 against runnable comparators: every metric, every baseline, every interval excluding zero. The gap is not an artifact of one workflow: Delphi recovers 51 cases the fusion baseline misses entirely while losing 19, and the exact McNemar test resolves the asymmetry at p=1.7×10^{-4}.

6 RQ2B: GENERALIZATION FINALS

Table 3: on SWE-bench localization Delphi leads MRR by $+0.274$ $[0.194, 0.412]$ over the strongest baseline and BCY@8k by $+0.251$ $[0.157, 0.433]$, while Recall@20 is a statistical tie ($+0.018$ $[-0.073, 0.075]$ vs. RRF): when twenty guesses are allowed, lexical engines eventually surface the file; Delphi's advantage is putting it first. The independent final shows the same shape at small n: Recall@20 beats BM25 ($+0.074$ $[0.019, 0.139]$) and lexical ($+0.046$ $[0.009, 0.083]$) and ties RRF (-0.009 $[-0.148, 0.083]$; any-gold 15 vs. 15). Nothing on either final shows a Delphi loss with an interval excluding zero.

7 FAILURE MODES THAT CHANGED THE PRODUCT

Two development findings moved code, and both survived their preregistered ablations into the frozen stack.

The engine returned the file the query quoted. By workflow, July any-gold@20 was 92% (trace2code) but 48% (comment2context): review-comment queries retrieved the markdown note or test that the comment already quotes, never the implementation behind it. The fix demotes paths the query envelope itself contains (only when evidence keys such as `comment` or `diff` are present) after the cross-encoder and before listwise. Development Recall@20 rose from 0.544 to 0.642 with the broader patch set ($+0.098$ $[0.041, 0.186]$).

The index silently dropped generated source. A locked SWE-adjacent gold path (`channelz.pb.go`, 114,500 bytes of protobuf-generated Go) was excluded by a global `*.pb.go` pattern in both old and new indexes—invisible until a per-gold-path searchability preflight failed. Agent-mode indexing now retains generated source under the existing 500,000-byte and NUL-content guards, the queued legacy worker enforces the same guards, and every confirmatory index passed an exact file/chunk/embedding audit (final corpus: 142 snapshots, 229,836 files, 1,089,836 chunks and embeddings, zero mismatches) plus the searchability preflight before any query ran. The frozen confirmatory commit is exactly the development winner plus this fix.

8 DOCUMENTATION: THE FAIR ORDEAL

Documentation retrieval is where hosted engines are strongest, and where output contracts differ most. The retrieval-plus-synthesis engine returns a fluent synthesized answer with citations; the documentation-only engine and Delphi return raw context. Scored naively, the synthesis engine leads: identifier hit 0.575 (full prompts) against Delphi's 0.400 (compact) and the documentation engine's 0.375 (quality-guided). Query shape is engine-specific, not a shared transform: a balanced, order-alternating ablation shows full prompts beat compact for the synthesis engine by 0.400 $[0.160, 0.640]$ on identifier hit, while a 220-request pinned-ID ablation shows the documentation engine statistically tied across shapes ($+0.010$ $[-0.085, 0.110]$).

Matching the contract. The synthesized-answer score conflates retrieval with the synthesis model's parametric knowledge—an answer can name `df.where` because the model knows pandas, not because retrieval found it. So we matched contracts: every engine's recorded retrieved context goes through the *same* frozen answer-style synthesis stage (`gpt-5.4-mini`, 1,600 output tokens, identical system prompt), plus a no-retrieval control with the same prompt shape. Three repeats per arm, 40 cases, zero technical failures (Table 4).

Three findings. First, the "documentation gap" was the contract, not the retrieval: under matched output contracts Delphi moves from 0.400 to 0.617 and sits above the synthesis engine's 0.575 ($+0.042$ $[-0.092, 0.183]$; 6 wins, 6 losses, 28 ties)—parity or better, with superiority unresolved. Second, the metric saturates: the bare model scores 0.492, so roughly half of *every* engine's synthesized-contract score is parametric knowledge, and the three engines finish within 0.05 of one another (Delphi vs. documentation engine -0.008 $[-0.117, 0.092]$). Identifier hit over synthesized text is a weak discriminator of retrieval; raw-context scoring remains the cleaner instrument. Third, the price differs: the synthesis engine's native pass averages 12.4 s per request against ≈ 5.1 s

Table 4: Matched-output-contract documentation comparison, 40 development cases × 3 repeats (the hosted synthesis engine's recorded pass is single-repeat by its own contract). Case-cluster bootstrap 95% intervals; every engine-vs-engine interval includes zero.

Arm	Identifier hit	Per-repeat	Δ vs. control
Documentation engine + synthesis	**0.625**	0.650/0.600/0.625	+0.133 [0.008, 0.258]
Delphi + synthesis	**0.617**	0.600/0.625/0.625	+0.125 [0.000, 0.250]
Synthesis engine (recorded, native)	0.575	—	+0.083
Model alone (no retrieval)	0.492	0.475/0.475/0.525	—

for Delphi's retrieval-plus-synthesis (2.6 s + 2.5 s means) and it is the only arm that never repeats byte-identically (§4).

9 RQ1: DOES ANY OF IT HELP AN AGENT?

Causal seeding (closed-tool agent). Track C freezes the policy model (`gpt-5.4-mini`) and the tool set (`list_dir`/`grep`/`read_file`/`submit`) and varies only the seed context block, 40 paired development cases per arm. File-F1: no seed 0.046; random files 0.013; BM25 seed 0.093; lexical seed 0.119; July-Delphi seed 0.234; oracle 0.858. The Delphi seed beats no-seed by +0.188 [0.055, 0.294] and BM25 by +0.142 [0.059, 0.208]; against lexical, final-any-gold improves by 0.225 (McNemar p=0.035) while the F1 interval narrowly includes zero [−0.002, 0.190]. Seeds causally change what a frozen agent reads and submits, ordered by seed quality; the ceiling (oracle 0.858) shows most of the remaining gap is retrieval, not the agent.

Direct generation utility (DS-1000). With the frozen generator, 40 cases × 3 repeats per condition: no retrieval 0.575; documentation engine 0.567 (−0.008 [−0.142, 0.125]); Delphi retrieval+synthesis 0.592 (+0.017 [−0.125, 0.158]); synthesis engine 0.642 (+0.067 [−0.075, 0.200]); head-to-head synthesis engine vs. Delphi +0.050 [−0.075, 0.183]. Every interval includes zero. On a 40-case development set with a strong generator, *no* context engine—ours included—demonstrably helps or hurts DS-1000 pass@1. Claims of downstream coding utility at this scale should be treated as unresolved by default.

Vignettes (not ARB JSON). Three questions a person would actually type, on pinned commits. Gin: why is my JSON HTML-escaped? Delphi, the synthesis engine, and grep all hit `render/json.go` (5.9 s / 12.0 s / 0.03 s). Tokio: does a dropped `JoinHandle` keep running? All three hit (4.7 s / 13.1 s / 0.12 s). Pytest: where is assert rewriting implemented? Delphi and grep hit `src/_pytest/assertion/rewrite.py`; the synthesis engine wrote a correct-looking explanation from tests and docs without returning the file (5.7 s / 16.9 s / 0.15 s). A fluent answer can hide a miss; that is why Track C grades submissions, not prose.

10 THE HOSTED REPOSITORY ARM, TRANSPARENTLY

The synthesis engine also markets a repository surface, and RQ2 deserves that head-to-head. It never became runnable. The development corpus requires 68 exact `(repo, base_commit)` snapshots in the provider's account. Whole-snapshot ingestion of large repositories (etcd, tokio, transformers) stalled in a non-terminal `syncing` state for over 24 hours; a prior evaluation round succeeded only by sharding snapshots into ≤100-file, ≤1.5 MB sources, and 32/68 required pairs remain terminally indexed from that round. Re-applying the sharded strategy, our minimal probe—one shard, the smallest unit the API accepts—was accepted by the sync API and then also failed to leave `syncing` within a 40-minute deadline; a multi-hour watcher recorded no late completion. Under the fail-closed coverage rule this is an external blocker, not a loss: no repository score is reported for that engine in either direction, and the head-to-head runs the moment 68/68 coverage is demonstrable.

11 WHAT WE CLAIM, AND WHAT WE DO NOT

Claimed. (1) On the untouched ARB round-3 final, Delphi beats whole-file BM25, the official lexical ranker, and their tuned fusion on all four metrics with repository-cluster intervals excluding zero. (2) On held-out SWE-bench localization it leads MRR and BCY decisively and ties Recall@20; on the independent final it leads or ties everywhere. (3) Under matched output contracts, Delphi's documentation retrieval plus a frozen synthesis stage is at parity or better with a hosted retrieval-plus-synthesis engine at less than half its latency. (4) Delphi's retrieval is near-deterministic in-process and restart-durable with a warm cache; the hosted engines are not close. (5) Retrieval seeds causally improve a frozen agent's submissions, ordered by seed quality.

Not claimed. (1) State of the art: the hosted repository head-to-head is externally blocked at 32/68 coverage, and our claim rule requires the complete comparable set. (2) Documentation superiority: $+0.042$ has an interval through zero; parity-or-better is the supported sentence. (3) Downstream DS-1000 utility, for anyone, at $n{=}40$. (4) Anything from the rejected file-level Okapi candidate, which failed its preregistered gates and ships default-off. (5) The resurrected July "95% DS-1000" number, which predates this protocol and stays retired.

12 CONCLUSION

Context engines help when they promote the file the query is about rather than the file the query already quoted, when they index the generated source the gold path lives in, and when they return the same answer twice. Making those three sentences true required a measurement protocol more than a modeling idea: untouched finals, exact hosted accounting, matched output contracts, cluster-aware intervals, and preregistered gates that rejected one of our own favorite features. The result is an engine that decisively beats the strongest runnable baselines on its home surface, matches a hosted synthesis engine on the surface where it was supposedly behind, and holds the only determinism story in the comparison—with the one remaining head-to-head blocked outside our code and recorded as exactly that.

AI USE STATEMENT

Generative AI tools were used to help formulate experimental questions, write and test evaluation and product code, operate experiments, inspect failures, search for related work, and draft and edit this manuscript. They were not used to generate benchmark gold labels. Empirical claims are checked against persisted run artifacts and canonical repository snapshots; code changes are tested and ablation-tested before inclusion. All AI-assisted work remains subject to human review before submission, and the authors take responsibility for the final content, claims, code, and artifacts.

ETHICS STATEMENT

The study uses public open-source repositories and public benchmark records. Repository contents are processed locally except where a named hosted retrieval or model API is itself the evaluated system. We retain commit identifiers and public issue or review text for provenance, but do not intentionally collect private repository data or credentials. Because benchmark results can be used in product marketing, we predeclare the claim rule, preserve failed runs, report exposure, and give losses the same prominence as wins.

REPRODUCIBILITY STATEMENT

All confirmatory numbers derive from one frozen commit with response-level serving-configuration attestation. Paired analyses use cluster bootstraps with 20,000 resamples at seed 1042 and exact McNemar tests. Hosted runs carry exact attempt ledgers; documentation comparisons pin resolver-selected library IDs. Per-case artifacts (queries, gold paths, ranked outputs, synthesized answers, latencies, and analysis JSON) are retained for every table in this paper, including failed and rejected runs.

REFERENCES

Zhaoling Chen, Robert Tang, Gangda Deng, Fang Wu, Jialong Wu, Zhiwei Jiang, Viktor Prasanna, Arman Cohan, and Xingyao Wang. Locagent: Graph-guided llm agents for code localization. In *ACL*, pp. 8697–8727, 2025. doi: 10.18653/v1/2025.acl-long.426.

Luyu Gao, Xueguang Ma, Jimmy Lin, and Jamie Callan. Precise zero-shot dense retrieval without relevance labels. In *ACL*, 2023.

Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can language models resolve real-world GitHub issues? In *ICLR*, 2024.

Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *EMNLP*, 2020.

Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *SIGIR*, 2020.

Han Li, Letian Zhu, Bohan Zhang, Rili Feng, Jiaming Wang, Yue Pan, Earl T. Barr, Federica Sarro, Zhaoyang Chu, and He Ye. Contextbench: A benchmark for context retrieval in coding agents. *arXiv preprint arXiv:2602.05892*, 2026.

Bowen Qin and Yi Xie. Agent retrieval bench: Evaluating repository context retrieval for coding agents. *arXiv preprint arXiv:2607.24882*, 2026.

Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: Bm25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

Chunqiu Steven Xia, Yinlin Deng, Soren Dunn, and Lingming Zhang. Agentless: Demystifying llm-based software engineering agents. *arXiv preprint arXiv:2407.01489*, 2024.

John Yang, Carlos E. Jimenez, Alexander Wettig, Kilian Lieret, Shunyu Yao, Karthik Narasimhan, and Ofir Press. Swe-agent: Agent-computer interfaces enable automated software engineering. In *NeurIPS*, 2024.

Fengji Zhang, Bei Chen, Yue Zhang, Jacky Keung, Jin Liu, Daoguang Zan, Yi Mao, Jian-Guang Lou, and Weizhu Chen. Repocoder: Repository-level code completion through iterative retrieval and generation. In *EMNLP*, pp. 2471–2484, 2023. doi: 10.18653/v1/2023.emnlp-main.151.

Fuwei Zhang, Yanzhao Zhang, Mingxin Li, Dingkun Long, Lexiang Hu, Pengjun Xie, Zhao Zhang, and Fuzhen Zhuang. Core-bench: A comprehensive benchmark for code retrieval in the era of agentic coding. *arXiv preprint arXiv:2606.11864*, 2026a.

Shaoqiu Zhang, Yuhang Wang, Jialiang Liang, Yuling Shi, Wenhao Zeng, Maoquan Wang, Shilin He, Ningyuan Xu, Siyu Ye, Kai Cai, and Xiaodong Gu. Swe-explore: Benchmarking how coding agents explore repositories. *arXiv preprint arXiv:2606.07297*, 2026b.

Sheng Zhang, Yifan Ding, Shuquan Lian, Shun Song, and Hui Li. Coderag: Finding relevant and necessary knowledge for retrieval-augmented repository-level code completion. In *EMNLP*, pp. 23278–23288, 2025. doi: 10.18653/v1/2025.emnlp-main.1187.